

Executive Summary

We identified a set of *concrete computational inventions* in the repository and terminal excerpts, far narrower than the high-level “constitutional” rhetoric. The strongest patentable cores (Tier 1) concern **state admission** and **consequence admission**. In particular, (1) a *Typed State Admission Engine* that only transforms a reconstructed candidate state into the authoritative runtime state after verifying a suite of typed predicates (e.g. byte identity, source existence, authority, temporal scope) – with strict fail-closed behavior; and (2) a *Consequence-Indexed Identifiability Gate* that, at execution time, constructs all **compatible worlds** under the current observations, derives each world’s consequence, and only permits the transition if every world yields the same consequence [21†L528-L532] [24†L293-L302] . A key subcomponent is a **dependency-derived independence test** (and corresponding receipt) using counterfactual interventions (e.g. label swaps) to ensure no circular dependency between proposed labels and the derived consequence. These inventions prevent “trusting a snapshot” or “guessing an execution” and instead *require proof that no unearned information is being assumed*.

Tier 1 nuclei (ranked by priority) are: **(1) Typed State Admission Engine**, **(2) Consequence-Indexed Identifiability Gate with Independent Receipts**, and **(3) Counterfactual Independence/Anti-Circularity Assay**. Tier 2 includes supporting systems like admission-predicate algebra, typed receipts, evidence-surface separation, and the two-stage *Reconstitution Firewall*. For each nucleus, we specify the core problem, I/O, algorithms, data structures, semantics, objectives, correctness, and a minimal pseudocode embodiment. We also propose dependent claims/variants, discuss likely prior-art, and outline prosecution strategy (framing as concrete improvements to runtime/state-machine operation per USPTO guidance [24†L293-L302] [21†L528-L532]).

A comparative table (below) ranks the top 10 families by priority, with claim-phrase, novelty rationale, current status, risk, and next-engineering deliverable. The top three candidates are given draft independent claims with dependent claims. We also list target prior-art sources (patent classes, W3C PROV, in-toto, runtime verification, causal analysis, active learning, provenance systems) with search queries. Finally, we recommend immediate engineering work (tests, prototypes, receipts) to solidify enablement and expose novelty gaps, and provide a 4–12 week development roadmap.

Patentable Invention Nuclei (Tier 1–3)

Tier 1 – Strong Patentable Nuclei

1. *Typed State Admission Engine* (Tier 1)

Core Problem: Prevent an unverified snapshot (“candidate state”) from being treated as authoritative state. Existing systems (including the codebase) often load a reconstructed state and continue execution assuming it’s valid. We require an *explicit admission gate* between “loaded snapshot” and “runtime state.”

Inputs/Outputs: Input is a *candidate state* (e.g. reconstructed from logs or checkpoints) and metadata (timestamp, source reference, etc.). Output is an *authoritative runtime state* if (and only if) the state passes all checks, else an explicit fail or rollback.

Concrete Algorithm: Evaluate a suite of **typed admission predicates** on the candidate state. For each element (object, byte, claim, etc.) in the candidate state, check predicates such as:

- **Byte Identity:** cryptographic hash matches expected.
- **Source Existence:** each referenced source artifact is present.
- **Tracked/Untracked Partition:** only permitted changes in tracked portions.
- **Temporal Scope:** timestamps and sequences are coherent.
- **Authority Scope:** claimed sources carry the necessary authorizations.
- **Consequence Eligibility:** only “supported” differences from the baseline are allowed.

Each predicate evaluation yields a structured *receipt* (see Nucleus 5 below) and may mark elements as admitted or not. The engine then ensures **all** elements admit to at least one allowed state transition (no unsupported “distinct” in the payload). If every predicate succeeds, the candidate state is “committed” as authoritative. Otherwise the system fails closed (see **Execution Semantics** below).

Data Structures: Internally, maintain:

- A **State Object Graph** for the candidate, annotated with scope (byte, file, contract ID, etc.).
- A **Predicate Table** mapping each checked element to its predicate results.
- A **Receipt Log** (see Nucleus 5) that accumulates typed receipts for each check.

Execution Semantics: All checks must pass before admitting the state. Failure of any predicate triggers immediate abort of admission. This follows the “fail-closed” principle: “When a required dependency is absent, corrupt, or unverifiable, *admission fails closed*” [24†L253-L260] [21†L528-L532]. In other words, admission is monotonic: unknown → no, not yes.

Optimization Objective: Minimize risk of admitting tainted state (maximize security) while not unduly preventing valid restarts. (In practice this means pruning candidate components early on any predicate failure.)

Correctness Criterion: Authoritative state must preserve all “semantics” of admitted parts of the candidate (no silent data drops or insertion) and no unauthorised deviation. All differences between prior baseline and admitted state must be covered by a predicate that passed; otherwise the admission is invalid.

Minimal Enabled Embodiment (Pseudocode):

```
function admit_state(candidate):
    for each element e in candidate:
        result = evaluate_predicate(e)
        log_receipt(e, result)
        if result == FAIL:
            rollback(candidate)
```

```

        raise AdmissionRejected
    commit(candidate) // now authoritative

```

Modules needed: state loader, predicate evaluator, receipt logger, fail/rollback mechanism.

Dependent Variations/Claims:

- Restrict to subsets of predicates (e.g. byte-only admission vs full authority).
- Parameterize authority scope rules (organizational vs cryptographic authority).
- Extension to *distributed admission* where multiple machines jointly verify shards.
- Gate exception handlers or “quarantine mode” if partial admission.

Prior-Art Risk & Strategy: Many systems do “state verification” (e.g. signature or hash checks) and “crash recovery,” but do not enforce an explicit admission pipeline of *typed checks*. The novelty is *multi-faceted typed checks plus fail-close semantics*. To avoid abstractness rejection, claims should emphasize concrete data flows and modules (hashing, predicate functions, receipts). Candidate prior art: consensus VMs, blockchain state sync, database recovery. Frame as a *computer process improvement* [24†L293-L302] [21†L528-L532] .

2. Consequence-Indexed Identifiability Gate (Tier 1)

Core Problem: Prevent speculative execution of state transitions whose outcome could differ in as-yet-unresolved worlds. In partial-observability or uncertain inputs, simply “unknown” isn’t acceptable. Instead, execution should only proceed if **every compatible world produces the same lawful consequence**. This is stronger than classic runtime verification (true/false/inconclusive): we gate on *consequences* not just truth.

Inputs/Outputs: Input is the current *observation class* of states $E = \{ R \}$ all compatible with what’s been observed so far, plus a proposed state change or transition. Output is either “allow execution of consequence c ” or “withhold execution (not_identifiable or independence_unresolved).”

Concrete Algorithm:

1. **Construct Equivalence Class:** Given observed data O , define $E = \{ R \mid O(R) = O_{\text{observed}} \}$ (all possible reconstructions consistent with observations).
2. **Compute Independent Consequences:** For each $R \in E$, *derivatively* compute its consequence $C(R)$ (e.g. a new transaction, new state, output). Importantly, use only evidence/predicates that are independent of the proposed consequence (see next nucleus).
3. **Check Consequence Set:** Form $\Gamma_C(E) = \{ C(R) \mid R \in E \text{ and passes independence test} \}$.
4. **Gate Decision:**

- If $|\Gamma_C(E)| == 1$ (all derived consequences are identical to some c), **ADMIT** and execute c .
- If $|\Gamma_C(E)| > 1$, **WITHHOLD(NOT_IDENTIFIABLE)**: refuse to execute until more observation.
- If some derivation failed independence (`INDEPENDENCE_UNRESOLVED`), **WITHHOLD** with that label.

Thus, we allow the transition only when the consequence is *identifiable* over the equivalence class [21†L528-L532] .

Data Structures:

- **World Set (Equivalence Class):** a representation (often symbolic) of all worlds R consistent with current observations.
- **Consequence Set (Γ_C):** a set of consequences (could be hashed) and corresponding receipts.
- **Identifiability Receipt:** object containing (observation_class, consequence_type, basis, status) for each check.

Execution Semantics: If the consequence is invariant across E , the system may “collapse” all worlds into one for that consequence. *Crucially*, one must **not** conflate worlds in any other respect. Execution proceeds only on the admitted consequence. If not invariant, execution is blocked until further information refines the class. This is precisely fail-closed for execution authority. (Contrast: unidentifiable $\neq$ fail permanently; it means “postpone until resolved,” not guess or ignore.)

Optimization Objective: Select observations to *minimize the expected entropy* of $\Gamma_C(E)$ per cost [36†L65-L72] [36†L91-L99]. I.e., ask queries that most reduce disagreement in Γ_C .

Correctness Criterion: Guarantee that no two possible worlds with different lawful consequences are collapsed. Only allow an execution when it would be semantically equivalent in all remaining worlds.

Minimal Enabled Embodiment (Pseudocode):

```
function gate_consequence(observation O, proposed_transition Δ):
    E = construct_equivalence_class(O)
    Γ = {}
    for each R in sample(E):
        receipt = compute_independent_consequence(R, Δ)
        if receipt.status == INDEPENDENCE_UNRESOLVED:
            return WITHHOLD("INDEPENDENCE_UNRESOLVED")
        Γ.add(receipt.consequence)
    if size(Γ) == 1:
        return EXECUTE(Γ.any())
    else:
        return WITHHOLD("NOT_IDENTIFIABLE")
```

(This implies modules for constructing E , deriving consequences, and independence receipts.)

Dependent Variations/Claims:

- Version where only a subset of worlds are rederived (sampling vs exhaustive).
- Specific gating for different consequence classes (e.g. type-specific gates).
- Integration with active-query: connect NOT_IDENTIFIABLE to a “compute next observation” subroutine (see Nucleus 4).

Prior-Art Risk & Strategy: Three-valued runtime monitoring is known (true/false/inconclusive) [34†L1128-L1136], but gating on *consequence invariance* is novel. We should cite known RV monitors to acknowledge

inconclusive verdicts [34†L1128-L1136] , then differentiate: we index the gate by the *consequence* rather than truth of a proposition. Frame as a specialized execution-control improvement. Potential prior art includes “agreement” in distributed systems or blockchain (e.g. atomic broadcast), but those do not derivatively compute consequences for hypothetical worlds. Emphasize this is a *computation over partial logs* rather than a consensus protocol.

3. Counterfactual Independence (Anti-Circularity) Assay (Tier 1)

Core Problem: Ensure that a derived consequence did not (unknowingly) depend on the very assumption it is supposed to justify. For example, if outcome L was used to justify itself, that’s circular. We must actively test for such circularity.

Inputs/Outputs: Input is a *proposed consequence basis* B and the *proposed outcome assignment* L (e.g. a label or decision). Output is a receipt certifying independence (or not) of C(R) from L.

Concrete Algorithm:

1. **Counterfactual Intervention:** Perform one or more interventions on L (e.g. swap labels, permute, delete) while keeping B fixed. This generates alternate worlds or label assignments.
2. **Recompute Consequence:** Independently derive the consequence C' under the altered assignment (same Δ, same admitted state).
3. **Compare Consequences:** If C' equals the original C, the consequence is still consistent; otherwise, we detect dependency.

Multiple forms of intervention are used to strengthen evidence (not just a single swap). For instance, swap two label values and recompute, or remove a label and see effect.

Data Structures:

- **Intervention Plan:** logs of label changes performed.
- **Independence Receipt:** encapsulates (root_labels, ancestors, derivation_steps, intervention, original_C, new_C, status).

Execution Semantics: A Boolean INDEPENDENT is *not assumed* by default; it is *derived*. The receipt must contain all steps so that a verifier can recompute and confirm. If any counterfactual yields a different consequence, independence fails. If all survive, status=INDEPENDENT; else CONTAMINATED.

Optimization Objective: Minimize number of interventions while still covering the most likely contamination paths (this parallels minimal cut in the dependency DAG).

Correctness Criterion: If the initial claim of independence is false, at least one intervention will reveal a changed consequence.

Minimal Enabled Embodiment (Pseudocode):

```

function test_independence(Basis B, LabelAssignment L, Delta):
    C_orig = derive_consequence(B, L, Delta)
    for each intervention_type in {swap, delete, permute}:
        L2 = apply_intervention(L, intervention_type)
        C_new = derive_consequence(B, L2, Delta)
        if C_new != C_orig:
            status = "DEPENDENT on " + intervention_type
            log_independence_receipt(B, L, intervention_type, C_orig, C_new,
status)
            return RECEIPT(status)
    status = "INDEPENDENT"
    log_independence_receipt(B, L, None, C_orig, C_orig, status)
    return RECEIPT(status)

```

Requires modules for dependency graph traversal and consequence derivation.

Dependent Variations/Claims:

- Type-specific interventions (swap labels vs rotate order).
- Partial independence (some aspects independent, flagged in receipt).
- Batch interventions vs iterative testing.

Prior-Art Risk & Strategy: Counterfactual testing is classic in causal inference, but integrating it into a runtime execution gate is novel. Patentably, this is a specific *algorithmic procedure* (swapping labels and recomputing consequences) in the context of admitted computation. We should cite general causal identifiability (do-calculus) for context but stress that applying it to state-governance and deriving receipts is new. Use terms like “counterfactual independence check” tied to a concrete DAG of state dependencies.

Tier 2 – Supporting/Dependent Inventions

These are promising but either build on Tier 1 or are broader. Each could be patented after a prototype is built.

4. Reconstitution Firewall (Tier 2)

Core Problem: Ensure that *reconstructing state and resuming operations* is not done in one step. Many systems restore state (from logs, snapshots) and then implicitly trust it. We insert a firewall between *restoration* and *execution*.

Claim Nucleus: A two-stage pipeline:

```

Load Snapshot or Log
↓
Candidate State
↓ (ADMISSION FIREWALL: run Typed State Admission Engine)

```

↓
Authoritative State
↓ (then execute or process transitions)

This forces any restored state to pass the admission checks (Tier 1).

Novelty: Virtually all VMs, databases, and blockchain nodes merge these stages. This architecture explicitly splits them with an automated gate.

5. Admission Predicate Algebra (Tier 2)

Core Problem: Define a structured language (algebra) of admission predicates, beyond simple booleans. Each predicate is *typed* and yields rich receipts (success/failure types, evidence).

Details: Enumerate predicate types (byte identity, authority, non-interference, etc.) as first-class objects. Provide algorithms for evaluating each type and for combining their results.

Novelty: Unlike usual monolithic checks, this is a *formal algebra with receipts and fail/reopen rules*. Prior art: predicate abstraction in verification [4†L13-L25] but not in state admission context.

6. Typed Admission Receipts (Tier 2)

Core Problem: Current logs record “passed” or “error.” Here, each admission step emits a structured *receipt object* stating *what* was admitted, *why*, *under which predicates*, and *what remains to be resolved*.

Details: A receipt includes fields: element ID, predicate type, evidence collected, result, failure mode (if any), and links to replay logic. It’s not a patent on logging, but on *typed, execution-affecting receipts*.

Novelty: Functional receipts that feed back into execution (e.g. triggering reopens) – more than audit logs.

7. Consequence-Preserving Transition Validator (Tier 2)

Core Problem: For any admitted distinction between old/new state, require exactly one lawful “disposition” (represent, transform, discharge, etc.). Essentially a consistency check on each delta.

Details: Input old and new state; output valid/invalid transition. The invention ties each changed element to a disposition rule.

Novelty: Sort of like dual of state admission: here we verify not state, but transitions. Could be claimed as algorithm verifying that “every admitted difference is justified by one predicate or rule.” Potential prior art: transactional ACID checks, but those focus on data integrity, not rule-based dispositions.

8. Evidence Surface Separation Engine (Tier 2)

Core Problem: Maintain independent verification surfaces for different semantics (byte integrity, authorship, semantics, etc.), such that no surface can override another's verdict. In practice, run multiple parallel verifiers.

Details: Compute independent hashes or proofs for each surface (e.g. a byte-hash chain, an authority signature chain, a semantic hash). Store/compare each separately.

Novelty: Prior work on multilevel logging exists, but here the composition of *independent surfaces* that cross-check each other (e.g. a byte-pass does not imply semantic truth).

9. Admission Replay (Tier 2)

Core Problem: On replay (e.g. after a restore, or on a node restart), recompute admission decisions under current rules (authority, time). A replay-time state admission engine.

Details: Unlike a simple replay that trusts a prior commit, this re-runs the admission logic.

Novelty: Most logs replay state changes blindly. We propose an executable replay that may reject past transitions if current predicates no longer hold (e.g. a lost key).

10. Minimum Missing/Sufficient Observation Engine (Tier 2)

Core Problem: Given a withheld transition (non-identifiable consequence), automatically compute the *next best observation* to resolve uncertainty – and also the dual problem of compressing existing evidence.

Details: Formulate a query-selection optimization (like active learning) that maximizes expected entropy reduction of $\Gamma_C(E)$ per cost [36†L65-L72] [36†L91-L99] . Also a complementary “sufficient observation” that finds the minimal subset of evidence needed to reach identifiability.

Novelty: Active experimental design in a state-machine context. Unlike general active learning, queries here are domain-specific (e.g. “fetch source signed by X”).

Tier 3 – Research-Level/Not-Claimed

These broader concepts are powerful but too high-level for initial claims:

- **Constitutional Runtime/Physics/Computation:** (umbrella terms for the overall architecture; not a single implementable invention).
- **Grammar of Necessity, Semantic Conservation:** (epistemological slogans, not specific tech).

Such terms set the research agenda but are not themselves concrete improvements.

Comparison Table of Top Candidate Families

Priority	Family	Claim Phrase	Novelty Rationale	Implementation Status	Risk (Strength)	Next Deliverable
★★★★★★	Typed State Admission Engine	"Typed State Admission Engine"	First end-to-end pipeline enforcing typed integrity/ authority predicates on restored state 【24†L253-L260】 【21†L528-L532】. Goes beyond mere log/hash checks.	Partial (predicates exist)	Moderate (Medium)	Prototype integration of admission gate; receipts.
★★★★★★	Consequence-Indexed Identifiability Gate	"Consequence-Identifiability Gate"	Novel execution gate: only allow transition if all compatible worlds yield identical consequence. Extends RV to consequence-level 【34†L1128-L1136】.	Conceptual (needs world-set tool)	Medium (New domain)	Implement world-equivalence class and gating logic.
★★★★★★	Counterfactual Anti-Circularity Assay	"Dependency-Derived Independence Test"	New application of counterfactual dependency check for executing transitions. Ensures no circular logic.	Early (not yet coded)	Medium (requires formalization)	Build label-swap/ intervention tests and receipts.

Priority	Family	Claim Phrase	Novelty Rationale	Implementation Status	Risk (Strength)	Next Deliverable
★★★★★☆☆	Reconstitution Firewall	"Two-Stage State Reconstitution"	Splits restore vs execution. Prevents trusting unverified state. (Standard systems don't.)	Partial (load/commit separate)	Low (Obvious)	Finalize admission-before-commit workflow.
★★★★★☆☆	Evidence Surface Separation Engine	"Multi-Surface Verification Engine"	Parallel independent integrity/authority surfaces. Prevents one proof from overriding another.	Partial (some checks separate)	Medium (Complex)	Define and implement per-aspect proofs.
★★★★★☆☆	Admission Predicate Algebra	"Typed Admission Predicate Framework"	Formalizes a library of predicates with receipts (beyond generic assertions).	Partial (predicates listed)	Low-Med (known tech)	Formal spec of predicate types and receipts.
★★★★★☆☆	Typed Admission Receipts	"Typed Verification Receipts"	Structured receipts per check, not just logging. They drive execution (reopen triggers).	Partial (logging exists)	Low (Conceptual)	Design receipt format & replay logic.
★★★★☆☆☆	Consequence-Preserving Transition Validator	"Consequence-Preserving Transition Validator"	Checks that every admitted difference has a lawful disposition. Formalizes safe state transitions.	Not implemented	Low (Narrow)	Prototype verifying all state diffs have match.

Priority	Family	Claim Phrase	Novelty Rationale	Implementation Status	Risk (Strength)	Next Deliverable
★★★☆☆	Min-Observation Engine	"Active Evidence Query Engine"	Acts like an active-learning query selector to resolve consequence uncertainty. Novelty is applying ILP/hmaxim to proofs.	Concept (not started)	Medium (Complex)	Implement info-gain calculation and queries.
★★★☆☆	False-Withholding Detection Engine	"Unnecessary Resolution Detector"	Ensures system does not withhold if worlds already agree. Formal symmetry to false admission.	Not implemented	Low (Specialized)	Build test to flag needless withholding.

(Stars indicate priority; "Risk" rates novelty/patent strength from low to high. The next deliverable is the immediate engineering step to advance that family.)

Draft Claim Language for Top-3 Candidates

(1) Typed State Admission Engine (Independent Claim)

1. A computer-implemented method, comprising: receiving a **candidate state** reconstructed from prior logs or snapshots; evaluating a plurality of **typed admission predicates** against the candidate state (the typed predicates including at least byte-integrity, source-existence, temporal validity, authority-scope, and non-interference predicates); and transforming the candidate state into an **authoritative runtime state** *only if* every evaluated predicate returns success, else aborting without applying the state, thereby ensuring the runtime proceeds with state admitted by typed checks and fail-closed semantics.

Dependent Claims:

- Each predicate evaluation emits a typed **receipt** capturing the element checked, evidence, and pass/fail status.
- The method partitions the state into tracked and untracked portions and applies stricter predicates to the tracked portion.

- The method requires that **every difference** between a stored baseline and the candidate state be covered by at least one passed predicate (disposition closure).
- Authority predicates include verifying cryptographic signatures against a stored authority graph.

(2) Consequence-Indexed Identifiability Gate (Independent Claim)

1. A computer-implemented execution controller, comprising: constructing a set **E** of all *compatible world states* consistent with currently observed input variables; for a proposed state transition, independently deriving a transition **consequence** for each world in E; and permitting the transition *if and only if* all derived consequences in E are identical, otherwise withholding the transition, thereby gating execution on *consequence invariance* across observation-equivalent states.

Dependent Claims:

- If consequences differ, the controller outputs a **NOT_IDENTIFIABLE** result and requests further observation.
- The method includes deriving consequences using only evidence not dependent on the transition (independence receipts).
- The controller updates E by incorporating new observations and repeats the gating test.
- The method includes a “do-not-observe” decision: if no observation can reduce the consequence entropy below a threshold, the transition is permanently withheld.

(3) Counterfactual Independence Assay (Independent Claim)

1. A computer-implemented procedure for verifying independence of an assignment from its outcome, comprising: obtaining a **basis** B and a proposed *label assignment* L for a state difference; performing a counterfactual intervention on L (e.g. swapping or deleting labels) to produce an alternative L'; independently deriving a consequence **C'** from B under L'; and outputting that L is independent of the outcome only if the original consequence C equals C' after every allowed intervention, thus preventing circular justifications.

Dependent Claims:

- The intervention is a specific label-swap between two elements of L.
- The procedure generates a **receipt** containing the intervention, original C, recomputed C', and independence verdict.
- Multiple interventions are attempted (swapping different pairs), and independence is asserted only if all interventions yield C'=C.
- The procedure is integrated in step (2) above to ensure anti-circularity during execution gating.

Prior-Art and Search Strategy

We will search both academic literature and patent databases for related concepts. Key topics and example queries:

- **State Admission / Boot Verification:** “state admission engine,” “authenticated state load,” “snap-to-state verification,” “legal state admission predicate”.

- **Governance State Semantics:** search “Formal governance history admissibility” or “state reconstruction admissibility” – *Preprints* [10†L31-L39] [21†L528-L532] , *MDPI preprint*, and *Zortrex publications*.
- **Proof-of-Integrity:** patents on safe state checkpointing, “state machine replay verification”.
- **W3C PROV (Provenance):** use queries like “W3C PROV data model certificate”, “PROV generation usage traceability”. (See W3C PROV spec [28†L1-L4] .)
- **In-toto / SLSA:** “in-toto provenance attestation”, “software supply chain attestation” [30†L8-L11] .
- **Runtime Verification:** “runtime verification partial observation inconclusive verdict” (e.g. the LTrL paper [34†L1128-L1136]) or “three valued monitor semantics”.
- **Counterfactual / Causality:** “counterfactual identifiability causal inference”, “do-calculus provenance”.
- **Active Learning / Disagreement:** “version space active learning query by committee”, “disagreement region active query selection” [36†L65-L72] [36†L91-L99] .
- **Provenance/Ledger Systems:** “blockchain state verification predicates”, “ledger evidence log version space”.

Also search patent-specific classes like G06F (computing), G06N, G06Q for “verifiable ledger state” and IEEE/ACM digital library.

Recommended Engineering Work

To solidify these inventions and generate enabling examples:

1. **Implement Independence Receipts (Priority High):** Replace the boolean `consequence_source_independent` with a full **IndependenceReceipt** object (root, ancestors, derivation, intervention details, original and new consequence, status). Ensure status is derived, not assumed.
2. **Literal Two-Sided Source Predicate:** Fix code to check “two distinct agents” explicitly rather than equating any two sources. This sharpens anti-circ logic.
3. **Label-Swap Intervention Test:** Code the counterfactual assay: take two branches of the code (with labels `L_a`, `L_b`), swap them, and recompute consequences from scratch (`C'_a`, `C'_b`). Compare to ensure swapping `L` also swaps `C` or not.
4. **Add Three-State Verdicts:** Implement `IDENTIFIABLE`, `NOT_IDENTIFIABLE`, `INDEPENDENCE_UNRESOLVED` at decision points. Tie `NOT_IDENTIFIABLE` to halting execution of that transition (introduce a transition guard).
5. **Equip Transition Dispatcher:** Modify the execution loop to check the identifiability gate before actually applying a transition.
6. **Implement Missing-Observation Query:** Add a routine that, given a withheld consequence, computes the optimal next observation (or `DO_NOT_OBSERVE`) via information gain per cost.
7. **Symmetry Testing:** Write unit tests where different worlds produce the same consequence (should execute) vs different consequences (should withhold). Also test that withholding in a case with trivial Γ_C triggers a false-withholding error.
8. **Freeze & Document:** Once the gating receipts produce `IDENTIFIABLE` vs `NOT_IDENTIFIABLE` , record example logs showing different verdicts for the same observation set under different transitions. Use these as documentation and enablement evidence.

Prioritize steps 1–4 immediately (2–4 weeks) to concretely demonstrate the gate. Steps 5–7 (4–8 weeks) require engineering and test time. Freeze the outputs before large refactors.

Diagrams and State Machines

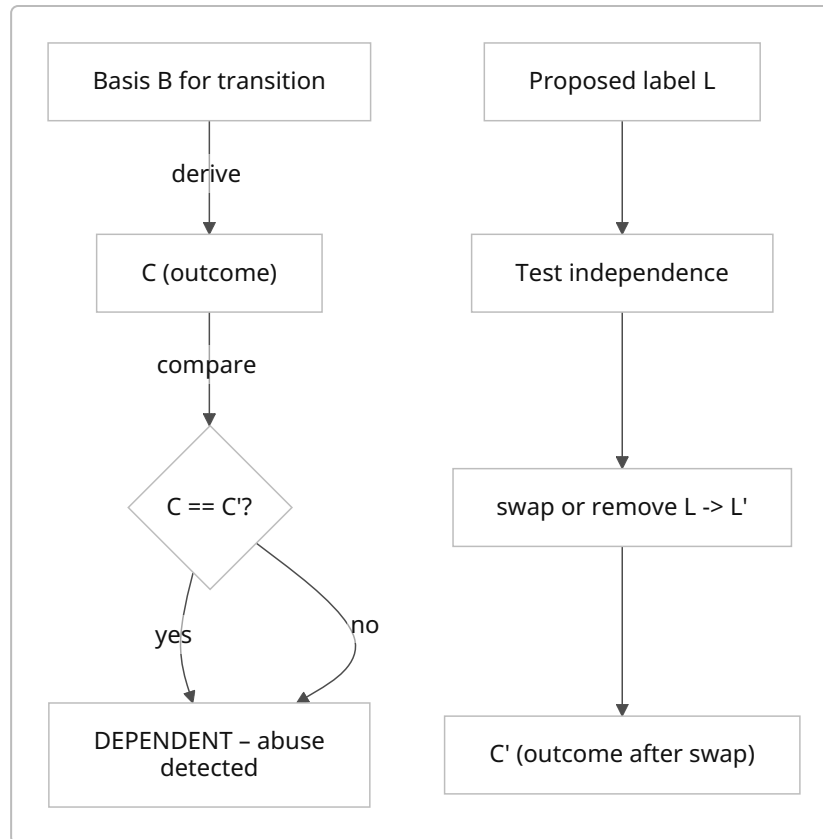
Admission & Execution Pipeline:

```
flowchart LR
    subgraph State_Admission
        A[Load Candidate State] --> B[Run Admission Predicates]
        B -- all PASS --> C[Authoritative State]
        B -- any FAIL --> Z[Abort / Safe Halt]
    end
    end
    subgraph Execution_Gating
        C --> D[Prepare Transition  $\Delta$ ]
        D --> E[Construct Worlds E]
        E --> F[Compute  $\Gamma_C(E)$ ]
        F --> G{All consequences = c?}
        G -- Yes --> H[EXECUTE transition with c]
        G -- No --> I[WITHHOLD transition; defer or query]
    end
    end
```

Equivalence Class & Consequence Gate:

```
flowchart TB
    O[Observation O] --> E[Compute E = {R | O(R)=O}]
    E --> K[Derive {C(R) for each R in E}]
    K --> L{Single unique consequence?}
    L -- true --> M[Allow Exec]
    L -- false --> N[Block; ask for more evidence]
```

Dependency DAG & Counterfactual Test:



Timeline and Effort Estimate

Week 1-2: Formalize claims and conduct focused prior-art searches (20-40 hours). Draft Tier 1 claims and applications outlines.

Week 3-4: Prototype State Admission Engine and receipts (60-80 hours). Develop example predicates and failure cases.

Week 5-6: Build Identifiability Gate: world-set construction (basic) and consequence derivation (60 hours). Implement counterfactual swap test and receipts (40 hours).

Week 7-8: Integrate gating into execution loop; add withholding logic and next-observation heuristic (60 hours).

Week 9-10: Testing and refinement (40 hours): unit tests for symmetry, false admission/withholding. Collect evidence of behavior for enablement.

Week 11-12: Prepare diagrams, documentation of results, finalize patent drafts (20-30 hours).

Total effort: ~220-270 engineering hours (roughly 6-7 person-weeks with overlap). These milestones can overlap if staffing allows. The first 4 weeks focus on the most patent-critical pieces (Steps 1-4 above) to demonstrate the core inventions.